

دانشگاه تهران
پردیس دانشکده‌های فنی
دانشکده مهندسی برق و کامپیوتر



یادگیری ماشین

تمرین شماره ۴

نام و نام خانوادگی
مجید صادقی نژاد

شماره دانشجویی
۸۱۰۱۰۱۴۵۹

۴ بهمن ۱۴۰۴

Contents

1	Question 1: One Full Iteration of K-Means	2
1.1	Assignment of Data Points to the Nearest Cluster Center	2
1.2	Computation of Updated Cluster Centers	2
1.3	New Cluster Centers	2
1.4	Final Cardinality of Cluster 0	3
2	Question 2: Linkage Strategies in Agglomerative Clustering	3
2.1	Computational Complexity Comparison	3
2.2	Robustness to Outliers	4
2.3	Complete Linkage Clustering and Dendrogram	4
3	Question 3: Dimensionality Reduction and Data Geometry	5
3.1	Linearly Correlated Gaussian Data	5
3.2	Two Concentric Circles	6
3.3	High-Dimensional Data with Strong Class Overlap	6
3.4	Data with Many Irrelevant or Noisy Features	7
4	Question 4: Formal Theory of LDA and Kernel PCA	7
4.1	Maximum Number of Discriminant Directions in LDA	8
4.2	Kernel PCA and the Covariance Operator in Feature Space	9
4.3	Representation of Eigenvectors	9
4.4	Derivation of the Kernel Eigenvalue Problem	10
4.5	Projection of a New Data Point	10
5	Question 5: Manual Computation of LDA	11

5.1	Class Means and Overall Mean	11
5.2	Within-Class Scatter Matrix	11
5.3	Between-Class Scatter Matrix	12
5.4	Generalized Eigenvalue Problem and Optimal Direction	12
5.5	Projection onto the Discriminant Direction	12
6	Question 6: Error Types	13
6.1	SFS, SBS, and Bidirectional Selection	13
6.2	Consequences of Greedy Behavior	15
6.3	Comparison with RFE and L1-Regularized Feature Selection	16
7	Question 7: PCA and Kernel PCA on Structured Data	17
7.1	Analysis of PCA Results	17
7.2	Comparison with Linear Discriminant Analysis (LDA)	17
7.3	Nonlinear Data and Kernel PCA	18
8	Question 8: Feature Selection and Visualization on a Real Dataset	19
9	Question 9: Image Segmentation	20

List of Figures

1	Data features Visualization	19
2	Features correlation matrix	20
3	Clustering using K-means	21
4	Clustering using GMM	21

In this assignment we will review some Clustering and Data Geometry topics and methods beside we'll check PCA and LDA and see them practically also we'll do some fundamental mathematics around these topics. In programming questions we Visualization a data set, analysis and check features of a dataset , and cluster a photo which is really interesting.

Question 1: One Full Iteration of K-Means

1.1 Assignment of Data Points to the Nearest Cluster Center

We compute the Euclidean distance from each data point to the initial centroids $\mu_0 = (5.3, 3.5)$ and $\mu_1 = (5.1, 4.2)$.

Point	Coordinates	$d(\cdot, \mu_0)$	$d(\cdot, \mu_1)$
x_1	(5.5, 3.1)	$\sqrt{0.2^2 + (-0.4)^2} \approx 0.447$	$\sqrt{0.4^2 + (-1.1)^2} \approx 1.17$
x_2	(5.1, 4.8)	$\sqrt{(-0.2)^2 + 1.3^2} \approx 1.32$	$\sqrt{0^2 + 0.6^2} = 0.6$
x_3	(6.6, 3.0)	$\sqrt{1.3^2 + (-0.5)^2} \approx 1.39$	$\sqrt{1.5^2 + (-1.2)^2} \approx 1.92$
x_4	(5.5, 4.6)	$\sqrt{0.2^2 + 1.1^2} \approx 1.12$	$\sqrt{0.4^2 + 0.4^2} \approx 0.57$
x_5	(6.8, 3.8)	$\sqrt{1.5^2 + 0.3^2} \approx 1.53$	$\sqrt{1.7^2 + (-0.4)^2} \approx 1.75$

Thus, the cluster assignments are:

Cluster 0 : $\{x_1, x_3, x_5\}$

Cluster 1 : $\{x_2, x_4\}$

1.2 Computation of Updated Cluster Centers

The new cluster centers are computed as the mean of the points assigned to each cluster.

1.3 New Cluster Centers

Cluster 0:

$$\mu_0^{(new)} = \left(\frac{5.5 + 6.6 + 6.8}{3}, \frac{3.1 + 3.0 + 3.8}{3} \right) = (6.3, 3.3).$$

Cluster 1:

$$\mu_1^{(new)} = \left(\frac{5.1 + 5.5}{2}, \frac{4.8 + 4.6}{2} \right) = (5.3, 4.7).$$

1.4 Final Cardinality of Cluster 0

After the update, Cluster 0 contains:

$$|\text{Cluster 0}| = 3 \text{ points.}$$

Question 2: Linkage Strategies in Agglomerative Clustering

2.1 Computational Complexity Comparison

In bottom-up (agglomerative) hierarchical clustering, both *single linkage* and *complete linkage* start with each data point as its own cluster and iteratively merge the closest pair of clusters until only one cluster remains.

- Single linkage: The distance between two clusters is defined as the *minimum* distance between any pair of points across the two clusters.
- Complete linkage: The distance between two clusters is defined as the *maximum* distance between any pair of points across the two clusters.

In a naive implementation, both methods require computing and updating an inter-cluster distance matrix at each merge step. For a dataset of n points:

$$\text{Time Complexity (both)} = \mathcal{O}(n^3), \quad \text{Space Complexity (both)} = \mathcal{O}(n^2).$$

Thus, single linkage and complete linkage have the same asymptotic computational complexity. The difference lies in the distance criterion, not in the order of growth.

2.2 Robustness to Outliers

Complete linkage is more robust to outliers.

Justification:

- Single linkage uses the minimum distance between clusters, which can cause a chaining effect, where clusters are merged due to a single close pair of points, even if the remaining points are far apart. An outlier can therefore connect distant clusters prematurely.
- Complete linkage uses the maximum distance between cluster members, ensuring that all points in the merged cluster remain relatively close. This prevents outliers from unduly influencing early merges.

Hence, complete linkage tends to produce more compact and well-separated clusters and is less sensitive to outliers.

2.3 Complete Linkage Clustering and Dendrogram

The dataset consists of the following points:

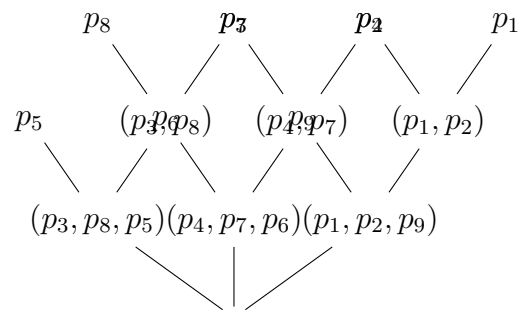
$$\begin{aligned} p_1(1, 1), \quad p_2(3, 2), \quad p_3(9, 1), \\ p_4(3, 7), \quad p_5(7, 2), \quad p_6(9, 7), \\ p_7(4, 8), \quad p_8(8, 3), \quad p_9(1, 4). \end{aligned}$$

Using complete linkage with Euclidean distance, the main merge steps are:

1. Merge p_1 and p_2 (smallest complete-link distance).
2. Merge p_4 and p_7 .

3. Merge p_3 and p_8 .
4. Merge p_5 with the cluster $\{p_3, p_8\}$.
5. Merge p_6 with the cluster $\{p_4, p_7\}$.
6. Merge p_9 with the cluster $\{p_1, p_2\}$.
7. Merge the left-side cluster $\{p_1, p_2, p_9\}$ with the upper-left cluster $\{p_4, p_7, p_6\}$.
8. Finally, merge all remaining clusters into a single cluster.

Dendrogram



This dendrogram illustrates the hierarchical merging process produced by complete linkage agglomerative clustering.

Question 3: Dimensionality Reduction and Data Geometry

3.1 Linearly Correlated Gaussian Data

Most appropriate method: PCA

Justification: Principal Component Analysis (PCA) seeks directions that maximize the variance of the data:

$$\max_{\mathbf{w}} \mathbf{w}^\top \Sigma \mathbf{w} \quad \text{s.t.} \quad \|\mathbf{w}\| = 1,$$

where Σ is the data covariance matrix.

For linearly correlated Gaussian data, the directions of maximum variance coincide with the principal axes of the Gaussian distribution. The geometry of the data is well captured by a linear subspace, and PCA efficiently removes redundancy caused by correlation while preserving most of the data variance. Since PCA makes no class-distribution assumptions and only relies on second-order statistics, it is well suited for this scenario.

3.2 Two Concentric Circles

Most appropriate method: Kernel PCA

Justification: Two concentric circles form a nonlinearly separable structure in the original input space. PCA fails in this case because any linear projection cannot unfold the circular geometry into a low-dimensional linear subspace. Kernel PCA overcomes this limitation by implicitly mapping the data into a high dimensional feature space via a nonlinear mapping $\phi(\cdot)$, and then performing PCA there:

$$K_{ij} = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle.$$

In this feature space, the concentric circles can become linearly separable or well represented by directions of high variance. The kernel trick allows Kernel PCA to capture nonlinear manifold structure, making it the most suitable choice for this geometry.

3.3 High-Dimensional Data with Strong Class Overlap

Most appropriate method: LDA

Justification: Linear Discriminant Analysis (LDA) is a supervised method that explicitly incorporates class labels. Its objective is to maximize the ratio of between-class scatter to within-class scatter:

$$\max_{\mathbf{W}} \frac{\det(\mathbf{W}^\top S_B \mathbf{W})}{\det(\mathbf{W}^\top S_W \mathbf{W})},$$

where S_B and S_W are the between-class and within-class scatter matrices.

In high-dimensional data with strong class overlap, variance alone (as used by PCA or Kernel PCA) is not necessarily informative for discrimination. LDA focuses on directions that best separate class means while minimizing intra-class variance. Even when classes overlap, LDA finds projections that maximize class separability, provided the class-conditional distributions are approximately Gaussian with similar covariances.

3.4 Data with Many Irrelevant or Noisy Features

Most appropriate method: LDA (if labels are available), PCA (if unlabeled)

Justification: Irrelevant or noisy features often increase variance without contributing to meaningful structure.

- PCA: PCA may fail when noise dominates the variance, since it blindly preserves directions of highest variance regardless of relevance. Noise can therefore be retained in the principal components.
- LDA: When class labels are available, LDA is more robust because it suppresses directions that do not contribute to class discrimination. Noisy features that do not affect class separation increase S_W without increasing S_B , and are therefore deemphasized.

Kernel PCA is generally not ideal in this setting, as nonlinear mappings can amplify noise and lead to overfitting unless carefully regularized.

Question 4: Formal Theory of LDA and Kernel PCA

4.1 Maximum Number of Discriminant Directions in LDA

Consider a classification problem with C classes and data dimension d . Let N_c be the number of samples in class c , $\boldsymbol{\mu}_c$ the mean of class c , and $\boldsymbol{\mu}$ the global mean.

The *between-class scatter matrix* is defined as

$$S_B = \sum_{c=1}^C N_c (\boldsymbol{\mu}_c - \boldsymbol{\mu})(\boldsymbol{\mu}_c - \boldsymbol{\mu})^\top.$$

Rank Argument Each term $(\boldsymbol{\mu}_c - \boldsymbol{\mu})(\boldsymbol{\mu}_c - \boldsymbol{\mu})^\top$ is a rank-1 matrix. Hence,

$$\text{rank}(S_B) \leq \min(d, C).$$

Moreover, the class means satisfy the constraint

$$\sum_{c=1}^C N_c (\boldsymbol{\mu}_c - \boldsymbol{\mu}) = \mathbf{0},$$

which implies that the vectors $\{\boldsymbol{\mu}_c - \boldsymbol{\mu}\}_{c=1}^C$ are linearly dependent. Therefore,

$$\text{rank}(S_B) \leq C - 1.$$

Linear Discriminant Analysis solves the generalized eigenvalue problem

$$S_B \mathbf{w} = \lambda S_W \mathbf{w},$$

where S_W is the within-class scatter matrix.

Since the number of nonzero generalized eigenvalues is upper-bounded by $\text{rank}(S_B)$, LDA can produce at most

$$\boxed{C - 1}$$

nonzero discriminant directions.

4.2 Kernel PCA and the Covariance Operator in Feature Space

Let $\phi : \mathbb{R}^d \rightarrow \mathcal{H}$ be a nonlinear mapping into a (possibly infinite- dimensional) Hilbert space $\mathcal{H}$. Assume the data are centered in feature space. The covariance operator in $\mathcal{H}$ is

$$\mathcal{C} = \frac{1}{N} \sum_{i=1}^N \phi(\mathbf{x}_i) \phi(\mathbf{x}_i)^\top.$$

Why Direct Eigen-Decomposition Is Infeasible The feature space $\mathcal{H}$ may have very high or infinite dimensionality, making it computationally infeasible to explicitly form $\mathcal{C}$ or compute its eigenvectors directly. Hence, standard PCA cannot be applied in $\mathcal{H}$ explicitly.

4.3 Representation of Eigenvectors

Let $\mathbf{v} \in \mathcal{H}$ be an eigenvector of $\mathcal{C}$ with eigenvalue λ :

$$\mathcal{C}\mathbf{v} = \lambda\mathbf{v}.$$

Substituting the definition of $\mathcal{C}$,

$$\frac{1}{N} \sum_{i=1}^N \phi(\mathbf{x}_i) \langle \phi(\mathbf{x}_i), \mathbf{v} \rangle = \lambda\mathbf{v}.$$

This shows that $\mathbf{v}$ lies in the span of $\{\phi(\mathbf{x}_1), \dots, \phi(\mathbf{x}_N)\}$, and hence can be written as

$$\mathbf{v} = \sum_{i=1}^N \alpha_i \phi(\mathbf{x}_i).$$

4.4 Derivation of the Kernel Eigenvalue Problem

Substitute $\mathbf{v} = \sum_{j=1}^N \alpha_j \phi(\mathbf{x}_j)$ into the eigenvalue equation and take inner products with $\phi(\mathbf{x}_k)$:

$$\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \alpha_j \langle \phi(\mathbf{x}_k), \phi(\mathbf{x}_i) \rangle \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle = \lambda \sum_{j=1}^N \alpha_j \langle \phi(\mathbf{x}_k), \phi(\mathbf{x}_j) \rangle.$$

Define the kernel matrix $K \in \mathbb{R}^{N \times N}$ with entries

$$K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle.$$

The above equation reduces to

$$K\boldsymbol{\alpha} = N\lambda\boldsymbol{\alpha}.$$

Thus, Kernel PCA reduces to solving the eigenvalue problem

$$\boxed{K\boldsymbol{\alpha} = N\lambda\boldsymbol{\alpha}},$$

where K is the centered kernel matrix.

4.5 Projection of a New Data Point

Let $\mathbf{x}$ be a new data point. Its projection onto the m -th principal component in feature space is

$$\langle \mathbf{v}_m, \phi(\mathbf{x}) \rangle = \sum_{i=1}^N \alpha_i^{(m)} k(\mathbf{x}_i, \mathbf{x}),$$

where $\boldsymbol{\alpha}^{(m)}$ is the eigenvector corresponding to the m -th largest eigenvalue.

Hence, both training and projection in Kernel PCA rely only on kernel evaluations and never require explicit computation in feature space.

Question 5: Manual Computation of LDA

5.1 Class Means and Overall Mean

The two classes are

$$\mathcal{C}_1 = \{(4, 2), (2, 4), (2, 3), (3, 6), (4, 4)\},$$

$$\mathcal{C}_2 = \{(9, 10), (6, 8), (9, 5), (8, 7), (10, 8)\}.$$

The sample means of each class are

$$\boldsymbol{\mu}_1 = \frac{1}{5} \sum_{\mathbf{x} \in \mathcal{C}_1} \mathbf{x} = (3, 3.8), \quad \boldsymbol{\mu}_2 = \frac{1}{5} \sum_{\mathbf{x} \in \mathcal{C}_2} \mathbf{x} = (8.4, 7.6).$$

The overall mean is

$$\boldsymbol{\mu} = \frac{1}{10} \sum_{i=1}^{10} \mathbf{x}_i = (5.7, 5.7).$$

5.2 Within-Class Scatter Matrix

The within-class scatter matrix is

$$S_W = \sum_{c=1}^2 \sum_{\mathbf{x} \in \mathcal{C}_c} (\mathbf{x} - \boldsymbol{\mu}_c)(\mathbf{x} - \boldsymbol{\mu}_c)^\top.$$

After computing the contributions from both classes, we obtain

$$S_W = \begin{pmatrix} 13.2 & -1.2 \\ -1.2 & 22.0 \end{pmatrix}.$$

5.3 Between-Class Scatter Matrix

The between-class scatter matrix is

$$S_B = \sum_{c=1}^2 N_c (\boldsymbol{\mu}_c - \boldsymbol{\mu})(\boldsymbol{\mu}_c - \boldsymbol{\mu})^\top, \quad N_1 = N_2 = 5.$$

Substituting the means,

$$S_B = \begin{pmatrix} 72.9 & 51.3 \\ 51.3 & 36.1 \end{pmatrix}.$$

5.4 Generalized Eigenvalue Problem and Optimal Direction

Fisher's criterion leads to the generalized eigenvalue problem

$$S_B \mathbf{w} = \lambda S_W \mathbf{w}.$$

Equivalently, we solve the eigenvalue problem for $S_W^{-1} S_B$. The nonzero eigenvalue is

$$\lambda \approx 7.63,$$

with corresponding eigenvector

$$\mathbf{w} = \begin{pmatrix} 0.909 \\ 0.417 \end{pmatrix},$$

(up to an arbitrary scaling factor).

This vector defines the **optimal LDA discriminant direction**.

5.5 Projection onto the Discriminant Direction

Each data point $\mathbf{x}$ is projected as

$$y = \mathbf{w}^\top \mathbf{x}.$$

Class 1 projections:

$$(4, 2) \rightarrow 4.47,$$

$$(2, 4) \rightarrow 3.49,$$

$$(2, 3) \rightarrow 3.07,$$

$$(3, 6) \rightarrow 5.23,$$

$$(4, 4) \rightarrow 5.30.$$

Class 2 projections:

$$(9, 10) \rightarrow 12.35,$$

$$(6, 8) \rightarrow 8.79,$$

$$(9, 5) \rightarrow 10.27,$$

$$(8, 7) \rightarrow 10.19,$$

$$(10, 8) \rightarrow 12.43.$$

Question 6: Error Types

6.1 SFS, SBS, and Bidirectional Selection

Sequential Forward Selection (SFS). Sequential Forward Selection is a wrapper-based feature selection method that starts with an empty feature set:

$$\mathcal{S}_0 = \emptyset.$$

At each iteration t , the algorithm evaluates all candidate features $f \notin \mathcal{S}_t$ by temporarily adding them to the current set and training the predictive model. The feature that yields the largest improvement in a predefined evaluation criterion (e.g., cross-validated accuracy, like-

likelihood, or loss reduction) is selected:

$$f^* = \arg \max_{f \notin \mathcal{S}_t} J(\mathcal{S}_t \cup \{f\}).$$

The selected feature is added to the set:

$$\mathcal{S}_{t+1} = \mathcal{S}_t \cup \{f^*\}.$$

The procedure continues until a stopping criterion is met, such as reaching a predefined number of features, no further improvement in the evaluation metric, or degradation in performance.

Sequential Backward Selection (SBS). Sequential Backward Selection operates in the opposite direction. It begins with the full feature set:

$$\mathcal{S}_0 = \{1, 2, \dots, d\}.$$

At each iteration, the algorithm considers removing each feature $f \in \mathcal{S}_t$ one at a time and evaluates the resulting subset. The feature whose removal leads to the smallest decrease (or largest improvement) in the objective function is eliminated:

$$f^* = \arg \max_{f \in \mathcal{S}_t} J(\mathcal{S}_t \setminus \{f\}).$$

The subset is updated as

$$\mathcal{S}_{t+1} = \mathcal{S}_t \setminus \{f^*\}.$$

Stopping criteria are analogous to SFS, for example when a desired subset size is reached or performance drops significantly.

Bidirectional (Stepwise) Selection. Bidirectional selection combines SFS and SBS. Typically, the algorithm starts like SFS by adding features one at a time, but after each forward step, it also considers removing previously selected features if their removal improves (or

does not significantly degrade) performance. Thus, the method alternates between forward inclusion and backward elimination, allowing limited correction of earlier decisions.

Greedy Nature. These methods are greedy because at each step they make the locally optimal decision adding or removing the single feature that yields the best immediate improvement without considering the global effect of future feature combinations. “Greedy” here means that decisions are irreversible (or only weakly reversible in bidirectional selection) and are based solely on short-term gains in the objective function.

6.2 Consequences of Greedy Behavior

The greedy nature implies that SFS, SBS, and bidirectional selection are not guaranteed to find the globally optimal feature subset. Even if the underlying learning problem is convex (e.g., linear regression with squared loss), the feature subset selection problem itself is combinatorial and non-convex.

Illustrative Scenario. Consider three features x_1, x_2, x_3 and a target variable y , where:

- x_1 and x_2 are weak predictors individually but strongly predictive when used together,
- x_3 is a moderately strong predictor on its own.

In SFS, the first step may select x_3 because it yields the highest individual performance. Once selected, adding either x_1 or x_2 may provide only marginal improvement, leading the algorithm to stop early or ignore them. The optimal subset $\{x_1, x_2\}$ is never explored. Similarly, in SBS, an early removal of a feature that appears redundant individually may prevent the algorithm from retaining a useful combination later.

Bidirectional selection mitigates this issue partially, but since it still evaluates only local, one-feature changes, it cannot guarantee recovery from poor early choices.

6.3 Comparison with RFE and L1-Regularized Feature Selection

Recursive Feature Elimination (RFE). RFE is a wrapper or embedded-like method that repeatedly fits a model and removes the least important feature according to model-specific importance measures (e.g., smallest absolute weight in a linear model).

- **Computational complexity:** Higher than SFS/SBS, as the model is refit many times.
- **Feature correlations:** Sensitive to correlated features; importance may be distributed across correlated variables.
- **Feature interactions:** Limited, as elimination is still based on marginal importance.
- **High-dimensional suitability:** Less suitable for very high-dimensional data due to repeated retraining.

L1-Regularization (Lasso). L1-regularized methods incorporate feature selection directly into model training by adding a sparsity-inducing penalty:

$$\min_{\mathbf{w}} \mathcal{L}(\mathbf{w}) + \lambda \|\mathbf{w}\|_1.$$

- **Computational complexity:** Efficient convex optimization; scales well to high dimensions.
- **Feature correlations:** Tends to select one feature from a group of correlated features and ignore the rest.
- **Feature interactions:** Does not capture interactions unless explicitly modeled.
- **High-dimensional suitability:** Highly suitable, especially when $d \gg n$.

7.1 Analysis of PCA Results

Explained Variance The first principal component (PC1) captures the largest amount of variance. This is inherent to the definition of PCA: it iteratively finds the orthogonal direction with the maximum variance in the data.

Class Separability The classes appear to be reasonably well separated, especially the Setosa class (Cluster 1). However, there is some overlap between Versicolor and Virginica in the 2D plane. Reasoning: PCA is an unsupervised method. Its objective is to maximize total variance, $\text{Var}(X_{\text{projected}})$, regardless of class labels. It does not try to maximize the separation between class means. The fact that they separate well indicates that the directions of maximum variance in the data also happen to be directions that differentiate the species (biologically, size variations correlate with species).

Appropriateness for Classification Is PCA appropriate for classification?

- Yes, but with caveats. It is often used as a preprocessing step to reduce dimensionality and noise, which can improve classifier performance (avoiding the curse of dimensionality).
- No, strictly speaking. Since it ignores labels, it might discard low-variance features that are actually highly discriminative for specific classes. For purely maximizing class separation, discriminant analysis is theoretically superior.

7.2 Comparison with Linear Discriminant Analysis (LDA)

If we used LDA instead of PCA, the projection would differ fundamentally:

1. Objective Function:

- PCA: Maximizes variation ($\mathbf{w}^T \Sigma \mathbf{w}$).

- LDA: Maximizes the ratio of between-class variance to within-class variance $(\frac{\mathbf{w}^T S_B \mathbf{w}}{\mathbf{w}^T S_W \mathbf{w}})$.

2. Conceptual Result:

- LDA computes directions that push the means of the classes as far apart as possible while making the spread of each class as tight as possible.
- The resulting plot would likely show the three classes much more tightly clustered and clearly separated, as the projection is explicitly optimized for this separability using the class labels y , which PCA ignored.

7.3 Nonlinear Data and Kernel PCA

Comparison: PCA vs. Kernel PCA

- Why Standard PCA Fails: Standard PCA is a linear technique. It searches for a linear projection (a rotation) of the data. Since the red and blue circles are concentric (nested), there is no straight line (or linear plane) that can separate them. Rotating the data with PCA does not change this topology; the classes remain nested in the 2D projection.
- Why Kernel PCA Succeeds: Kernel PCA allows us to perform PCA in a high dimensional feature space without explicitly computing the coordinates. Nonlinear Mapping: It implicitly maps the 2D data into a higher dimensional space where the classes become linearly separable. For concentric circles, an RBF kernel essentially maps the data onto a hill shape where the inner circle is at the peak and the outer circle is at the base. Linear Separation in Feature Space: A hyperplane in this high-dimensional space can easily slice between the peak and the base. Kernel Trick: The Kernel Trick ($k(x, y)$) allows us to compute the dot products required for PCA in this high-dimensional space using only the original 2D vectors, avoiding the intense computational cost of the explicit mapping. Result: The projection we see is the result of

this high-dimensional separation being projected back down, unfolding the nonlinear structure.

Question 8: Feature Selection and Visualization on a Real Dataset

Feature Selection Analysis on Heart Disease Dataset, In this assignment, we will perform feature selection using multiple techniques (Correlation based, Univariate, RFE, RFECV, Lasso) and compare their behavior, stability, and interpretability using the Heart Disease Dataset. We will also compare these with PCA.

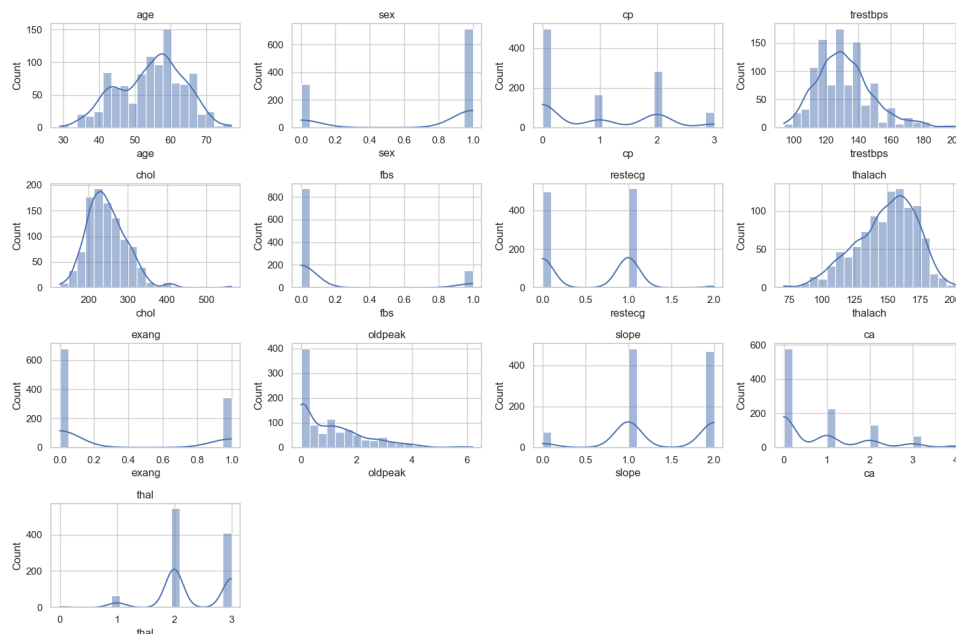


Figure 1: Data features Visualization

Final Discussion Predictive Performance: RFE and RFECV typically provide feature subsets that perform as well as or better than the full set by removing noise. Univariate methods are fast but may miss interactions. Interpretability Lasso and RFE indicate which specific clinical features drive the prediction. PCA transforms features, losing clinical meaning. Correlated Features: Lasso: Tends to pick one from a group of correlated features and zero out



Figure 2: Features correlation matrix

the others arbitrarily. RFE: Can handle correlations better but might still be unstable. Univariate: Ignores correlations entirely (ranks them individually). PCA: Handles correlations perfectly by orthogonalizing them, but loses feature identity. Stability: RFECV adds stability by validating the subset across folds, ensuring the selection isn't biased by a specific train/test split.

Question 9: Image Segmentation

In this assignment, we will segment two images into $K \in \{2, 3, 4, 5\}$ segments. The methodology we use is Feature Extraction: For each pixel, we construct a 5-dimensional feature vector: Normalized vertical coordinate ($r' \in [0, 1]$), Normalized horizontal coordinate ($c' \in [0, 1]$), Normalized grayscale intensity ($i' \in [0, 1]$), Normalized vertical coordinate ($r' \in [0, 1]$), Normalized horizontal coordinate ($c' \in [0, 1]$).

dinate ($c' \in [0, 1]$), Normalized Red channel ($R' \in [0, 1]$), Normalized Green channel ($G' \in [0, 1]$), Normalized Blue channel ($B' \in [0, 1]$) for Clustering we use K-Means: Minimizes Euclidean distance in the 5D space. GMM Fits Gaussian Mixture Models and assigns labels based on maximum posterior probability (MAP). Visualization: We map the resulting labels to distinct colors to visualize the segmentation.

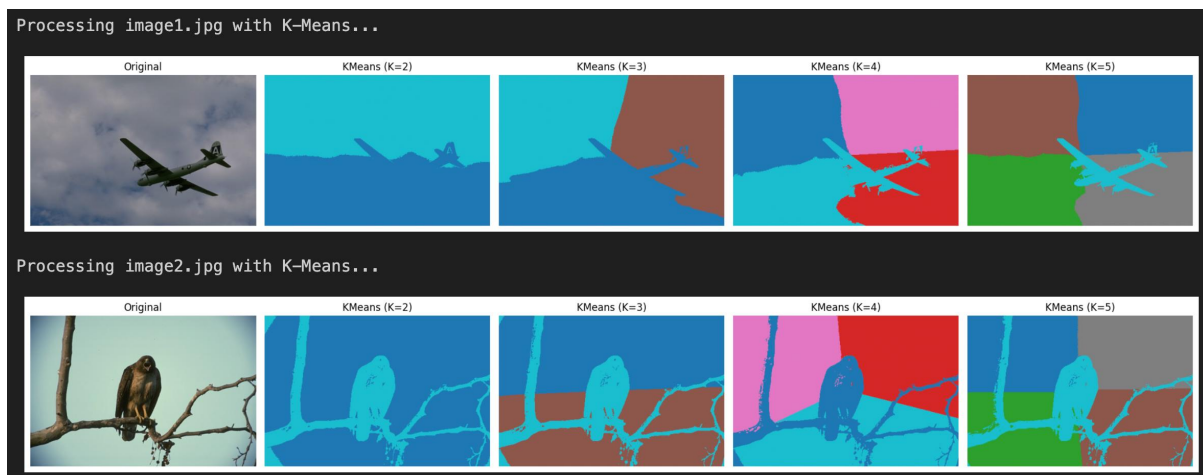


Figure 3: Clustering using K-means

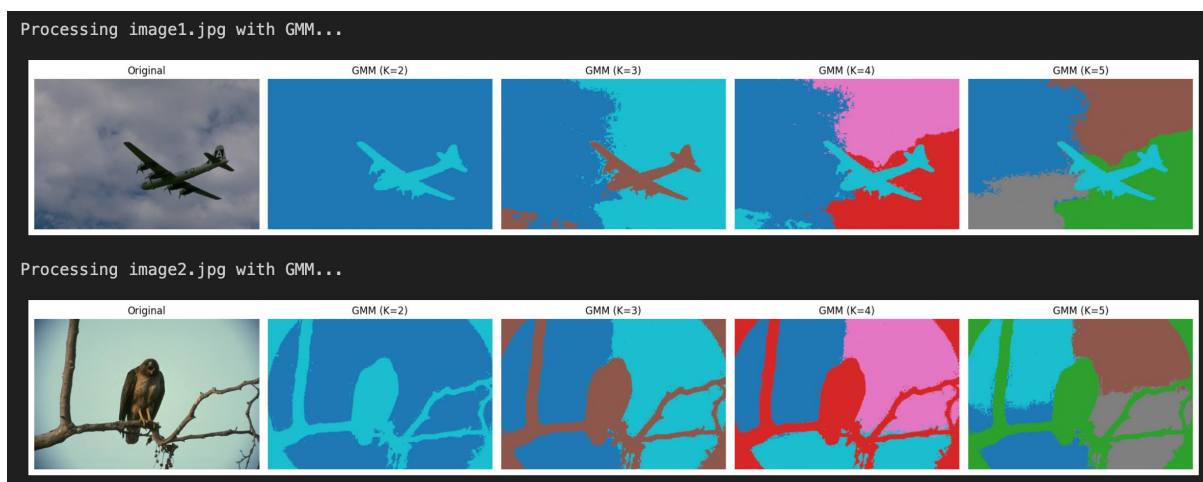


Figure 4: Clustering using GMM

Differences Observed Cluster Geometry (Spherical vs. Ellipsoidal): K-Means creates clusters that tend to be **spherical** in the 5D space. It treats color differences and spatial distance with fixed relative importance (determined by our normalization). It effectively groups pixels that are close in both color and position. GMM learns a covariance matrix for

each cluster. This allows clusters to be **ellipsoidal**. For example, a cluster in GMM can be stretched along the spatial axes but narrow in color space (representing a large region of uniform color), or vice versa. This often results in GMM capturing elongated structures (like the hawk's branch or the plane's wing) more naturally as single continuous segments even if they extend across the image. Boundary Smoothness: K-Means boundaries are essentially intersections of hyperplanes (Voronoi tessellation). GMM boundaries are quadratic curves (intersections of Gaussian densities). This often leads to smoother, more organic-looking boundaries in the segmentation map. Sensitivity to Variance: K-Means assumes all clusters have roughly equal variance. GMM allows one cluster to be very tight (e.g., specific distinct text or a sharp object) and another to be very broad (e.g., the background sky varies in diverse shades of blue). This adapts better to real images where the background often has high variance while the subject has lower variance.